

Systems Programming

2025/2026 - The Project – Part B



Space Trash



In a parallel universe, various planets face the perils of the trash roaming the space. These planets face doom if trash reaches a high level.

To respond to this threat, each planet decided to build and launch a **trash-ship** to collect the trash.

In this game, each user controls one of these **trash-ships** to collect trash and deposit it safely for recycling.

1 The Universe

The universe is facing extinction due to the trash that follows a set of special rules (some natural, some other anthropogenic):

- The universe is a square;
- The number of planets is fixed, and each one is named with a single letter;
- Periodically, new trash is generated in the universe;
- There are trash-ships trying to collect trash;
- Collected trash should be deposited in the recycling planet.
- If a **trash-ship** collides with a planet that is not the recycling, the whole trash cargo is spilled in the universe;
- When a trash hits a planet, new trash is generated;
- Trash is affected by gravity and has inertia;
- **trash-ships** are affected by gravity and inertia.

This parallel universe is ruled by laws similar to those of our own universe, but some other para-physical rules also apply.

1.1 Physics rules

Only trash and **trash-ships** are affected by gravitation force generated by the planets and the planets are stationary. The following constants should be used to calculate forces, acceleration, velocity, and location of trash:

- planet mass – 10 mass units
- trash mass – 1 mass units

- distance units – 1 pixel
- time unit – 10ms
- gravitational constant – 1
- trash friction – reduces velocity by 1% in every time unit
- trash-ship mass – 1 mass unit regardless on its cargo
- trash-ship friction – reduces velocity by 1% in every time unit

The code to calculate the gravitational force for each trash/trash-ship is the following:

```
void new_trash_acceleration(planet_stucture planets[], int total_planets,
                           trash_stucture trash[], int total_trash){
    vector total_vector_force;
    vector local_vector_force
    for (int n_trash = 0; n_trash < N_TRASH; n_trash++){
        total_vector_force.amplitude = 0;
        total_vector_force.angle = 0;
        for (int n_planet = 0; n_planet < N_PLANETS; n_planet++){
            float force_vector_x = planets[n_planet].x - trash[n_trash].x;
            float force_vector_y = planets[n_planet].y - trash[n_trash].y;
            local_vector_force = make_vector(force_vector_x, force_vector_y);
            local_vector_force.amplitude = planets[n_planet].mass /
                                           pow(local_vector_force.amplitude, 2);
            total_vector_force = add_vectors(local_vector_force,
                                              total_vector_force);
            trash[n_trash].acceleration = total_vector_force;
        }
    }
}
```

The velocity and position can then be computed using the following functions:

```
void new_trash_velocity(trash_stucture trash[], int total_trash){
    for (int n_trash = 0; n_trash < total_trash; n_trash++){
        trash[n_trash].velocity.amplitude *= 0.99;
        trash[n_trash].velocity = add_vectors(trash[n_trash].velocity,
                                              trash[n_trash].acceleration);
    }
}

void new_trash_position( trash_stucture trash[], int total_trash){
```

```

for (int n_trash = 0; n_trash < total_trash; n_trash++){
    trash[n_trash].x += trash[n_trash].velocity.amplitude *
                        cos(trash[n_trash].velocity.angle);
    trash[n_trash].y += trash[n_trash].velocity.amplitude *
                        sin(trash[n_trash].velocity.angle);;
    correct_position(&trash[n_trash].x);
    correct_position(&trash[n_trash].y);
}
}

```

By calling these three functions every 10ms, it is possible to simulate the basic gravitational mechanics of this universe.

These three functions can also be applied to trash-ships.

1.2 Para-physics rules

When a trash or **trash-ship** reaches the edge of the universe, it is teleported to the opposite side at the same speed (amplitude and direction).

Although planets have a radius of 20, trash can enter the interior of the planet, and continue moving as if the planet is a single point.

When the trash touches the center of the planet (distance between centers lower than 1.0), a new trash emerges in a random place of the universe.

When the amount of trash roaming the universe reaches a limit, the whole universe collapses and is destroyed.

When there are no trash-ships cleaning the universe there is no new trash being generated:

- humanity temporarily stops creating trash;
- trash that hits a planet does not generate new trash.

1.3 Anthropic rules

Trash, ships, and recycling planets also have some rules:

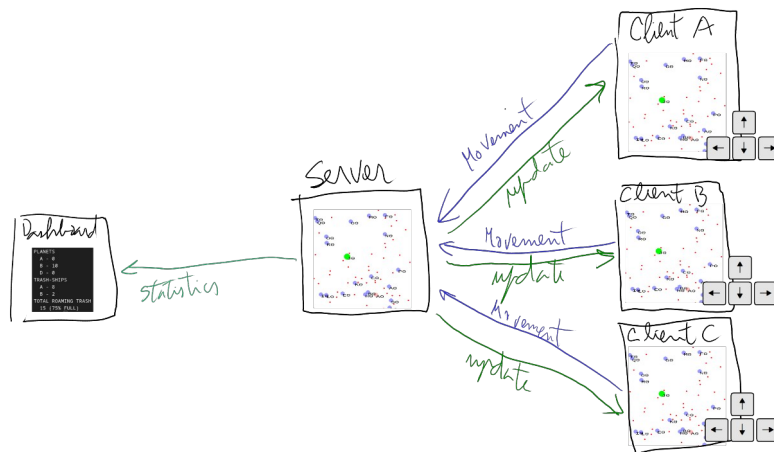
- Periodically and when there are trash-ships cleaning the universe, humanity generates trash that appears in random places in the universe.
- Each planet has one **trash-ship** that is operated by a human. These ships try to collect all roaming trash before the universe collapses.
- Trash is collected when the ship comes into contact with it.
- Ships have a limit on the capacity of collected trash. Before reaching that limit, collected trash needs to be deposited on the recycling planet.
- At any given time, only one planet can recycle trash collected by **trash-ships**.
- If a **trash-ship** hits a non-recycling planet, all of the trash cargo is spilled in the universe.

- The planet capable of recycling trash is randomly assigned and changes periodically.

2 Project – Part B

In the second part of the project, students will implement a distributed system composed of three communicating components/applications:

- One server that simulates the whole universe (**new-universe-server**);
- Clients that allow users to control **trash-ship** (**new-trash-ship-client**).
- One control dashboard that displays the statistics of the entire universe (**new-dashboard**).



2.1 new-universe-server

The **new-universe-server** will replace the **universe-simulator** and **universe-server** implemented in part A of the project, by integration all their functionalities. Besides all the previously implemented functionalities, this new component will implement the following:

- handle of various trash-ships;
- application of the physics rules (described in section 1) to **trash-ships**;
- periodic generation of trash;
- periodically change of the planet capable of recycling trash.

The operation of the **new-universe-server** will be as follows:

- initialization of the universe with planets and trash
- sockets creation
- concurrently
 - handling of clients
 - client connections
 - movement order from one client

- universe update (apply physics rules to trash and ships)
- universe state communication to clients
- universe display
- verification of end of the universe

2.2 new-trash-ship-client

The **new-trash-ship-client** will replace the **universe-client** implemented in part A of the project, with the following added functionality:

- display of the universe state

The **new-trash-ship-client** will connect to the **new-universe-server**, receive information about the trash-ship being controlled, forward movement commands to the server, and display the universe. The display of the universe in the **new-trash-ship-client** should show the changes of all the planets, trash, and trash-ships as they happen in the server.

Concurrently, this client should read the user keys, send them to the **new-universe-server**, and also receive all updates to the universe state and display them in an SDL window. The **new-trash-ship-client** SDL window should display exactly the same as the **new-universe-server** window.

2.3 new-dashboard

The **new-dashboard** is a completely independent application that will receive all information about trash-ships, planets, and overall trash and print it on the screen:

- for each planet, the amount of recycled trash-ship
- for each trash-ship, the current trash cargo
- total amount of roaming trash and proximity to the end of the universe

This application can simply print on the screen the information it receives from the **new-universe-server**, not requiring the use of the SDL library.

```

PLANETS (Recicled trash)
A - 0
B - 10
D - 0
TRASH-SHIPS (Trash Cargo)
A - 8
B - 2
UNIVERSE
ROAMING TRASH 15
TRASH CAPACITY 75%
```

This application should be developed in a language different from C (for instance, python or java).

Example of the **new-dashboard** output with three planets, two trash-ships and a maximum amount of trash of 20:

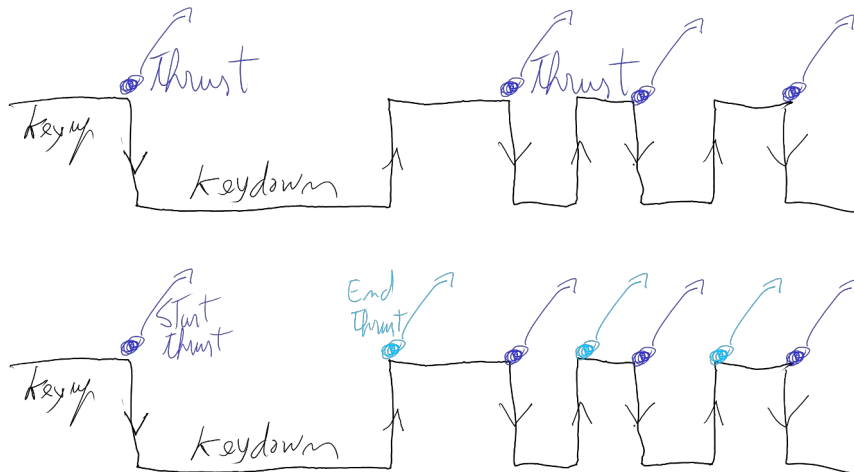
2.4 trash-ship movement

In the universe-server (from part A), no physics rules were being applied to trash-ships, as such it was supposed that the trash-ships would only move when the universe-client sent a movement message, and such messages would only affect the position of the trash ship, i.e. each key press would move the ship in a certain position, by a certain amount of space.

In the **new-universe-server** each trash-ship will have inertia and should have its velocity affected by:

- gravitational forces and accelerations (just like the trash);
- trash-ship friction (just like the trash);
- thrust generated by engines.

As such, the command given by the **new-trash-ship-client** should not affect the position of the ship, but should change its velocity vector. This thrust can be treated as instantaneous (when the user presses a cursor key) or continuous (during the period the user keeps the cursor key down).



The simplest solution is to send a thrust message whenever a user presses or releases a cursor key (SDL_KEYDOWN or SDL_KEYUP events). This thrust message should be translated into applying a force to the trash ship in a specific direction, thereby generating an acceleration and updating the velocity. To simplify, the mass of the ships should not affect the transformation of this force into acceleration/velocity/position. If no key is pressed, no movement/thrust commands are sent from the **new-trash-ship-client** to the **new-universe-server**, no thrust force is applied, and only the physics rules affect the trash-ship trajectory and velocity (like the trash).

Another more complex solution (not necessary to be implemented) is to use the cursor keys as an accelerator pedal, i.e., during the period that the key is pressed, a force is applied to the trash-ship. This solution requires

- the client to catch both SDL_KEYDOWN and SDL_KEYUP events;
- send two different messages equivalent to “pedal down”/“pedal up”;
- in the server the continuous calculation of the force “added by the engine”, while the key is pressed.

2.5 Periodic operations

The whole system (**new-universe-server** and **new-trash-ship-client**) is required to do some periodic operations with different periods/frequencies:

- Application of physics rules to trash and trash-ships – **10ms period** (100Hz)
- New trash Generation – **10s period** (0.1Hz)
- Change of planet capable of collecting trash – **30s period**
- Display update rate – **33ms period** (30Hz)

2.6 Universe limits

This universe has some limits:

- Number of planets
- Number of trash-ships
- Trash-ship capacity
- Maximum number of trash before universe collapses

When the amount of trash in the universe reaches the defined maximum, the universe collapses, and all clients should be notified. Students should define what happens next.

With respect to the maximum capacity of the trash ships, students should guarantee that the trash-ships do not collect any more trash until it is deposited on a planet.

If a client tries to connect and all trash-ships are in operation, he should be notified of it.

2.7 Configurations file

The three components need some configuration that should not be hard-coded, but provided as a configuration file with the following information:

- **new-universe-server**
 - Socket ports;
 - Number of planets;
 - Maximum number of trash before the universe collapses;
 - Initial number of trash;
 - Capacity of the trash-ships.
 - Dimensions of the universe;
- **new-trash-ship-client**
 - Sockets addresses and ports;
 - Dimensions of the universe;

- **new-dashboard**
 - Sockets addresses and ports.

2.8 Network disconnects

As opposed to pipes or regular sockets, ZMQ sockets do not inform the communicating entities if the other end of the sockets is still connected, but in the project students should:

- notify the users if the server was terminated (orderly or abruptly)
- remove a trash-ship if its new-trash-ship-client terminates (orderly or abruptly)

The orderly termination can be implemented with a special message. While a timeout can be used to detect if the other end of the socket is still alive.

3 Software architecture

Students should try to implement layers on this project and data abstraction. As such, for the universe simulator, there should be the following layers and C files.

new-universe-server	new-trash-ship-client	new-dashboard	new-universe-server.c new-trash-ship-client.c new-dashboard.c
SDL	SDL	Display	
Graphical display processing	Graphical display processing		Display.c display.h
	Keyboard processing		cursor_processing.c cursor_processing.h
Gravitational rules			physics-rules.c physics-rules.h
Data management	Data management		universe-data.c universe-data.h
Communication	Communication	Communication	Communication.c Communication.h

Some of these layers/files will be an evolution or reuse of those from part A of the project.

4 Development technologies

Besides Standard C, students should only use the following extra libraries:

- **SDL2** for displaying the universe (**new-universe-server** and **new-trash-ship-client**) and reading the cursor keys (**new-trash-ship-client**);
- **libconfig++** for reading the configuration files; (documentation provided in **libconfig** folder)
- **ZeroMQ TCP** socket for communication between components;
- **protocol-buffers** for the encoding of messages between components.

- **Threads** to allow concurrency in the **new-universe-server** and **new-universe-client**
- **Mutex/locks** to guarantee correct concurrent access to shared data

Students should implement the system using the C language **WITHOUT** using the following:

- select;
- non-blocking communication;
- active wait;
- signals.

4.1 Limitations of the technologies

ZeroMQ sockets are not thread safe, meaning that ZeroMQ sockets should not be shared between threads. In the implementation of the part B of the project, this requires that:

- only one thread receives and processes all the requests from clients (only one thread using a REQ sockets)
- only one thread sends all updates (using the PUB socket).

SDL also has some limitations about what thread can/should execute SDL functions:

- All functions related to windows creation and management should be called from the **main**
- All functions related to the drawing of the graphical window should be called from the **main**
- Functions that receive events (**SDL_PollEvent** or **SDL_WaitEvent**) can only be called from the **main**
- The **SDL_PushEvent** function can be called from any thread

this requires that the drawing of the windows to be done in the main, inside the vent processing loop.

5 Order of development

For success in the development (higher grade :), better results, and more learned knowledge, it is advised that users follow the presented order of development:

1. Change of name of universe-simulator (part A) into **new-universe-server**
2. Periodic generation of trash
3. Periodic change of the planet capable of collecting trash-ship
4. Implementation of the 30Hz display update rate
5. Change of the name of universe-client (part A) into **new-universe-client**
6. Integration of the trash-ships data-structures into **new-universe-server**
7. Integration of sockets and communication into **new-universe-server**
 - a) Acceptance of client connect
 - b) Movement of trash-ships

8. Implementation of physics rules on trash-ships
9. Implementation of the universe display in the **new-universe-client**
 - a) creation of a graphical window
 - b) transmission the universe state
10. Implementation of the **new-dashboard**
11. All other functionalities and verifications

6 Error treatment / Cheating

When implementing a distributed/network-based system, servers cannot guarantee that clients will lawfully adhere to the defined protocol.

If the communication protocol permits it, malicious programmers can exploit the devised messages and interactions for cheating.

In addition to verifying all messages received on the server to detect communication errors, the protocol and data exchanged between clients and the server should ensure that a malicious client cannot cheat by subverting the semantics and order of messages.

Here, we are not addressing hacking concerns that could be solved using cryptography. The code must ensure that programmers with a C compiler and knowledge of the protocol cannot disrupt the game (for instance, by moving other **trash-ships**, ...).

7 Project submission

The deadline for submitting for part B of the project will be **Friday, 9th January 2026 at 19h00** on FENIX.

Before submission, students should create the project group and register at FENIX.

Students should submit a unique **zip** file containing the code the components in suitable folders:

- **new-universe-server;**
- **new-trash-ship-client;**
- **new-dashboard.**

The students should also provide suitable Makefile(s) to compile the various programs.

8 Project evaluation

The grade for this project will be given taking into consideration the following:

- Number of functionalities implemented;
- Communication;

- Code structure and organization;
- Error validation and treatment;
- Cheating robustness;
- Comments.